Series Q Q2

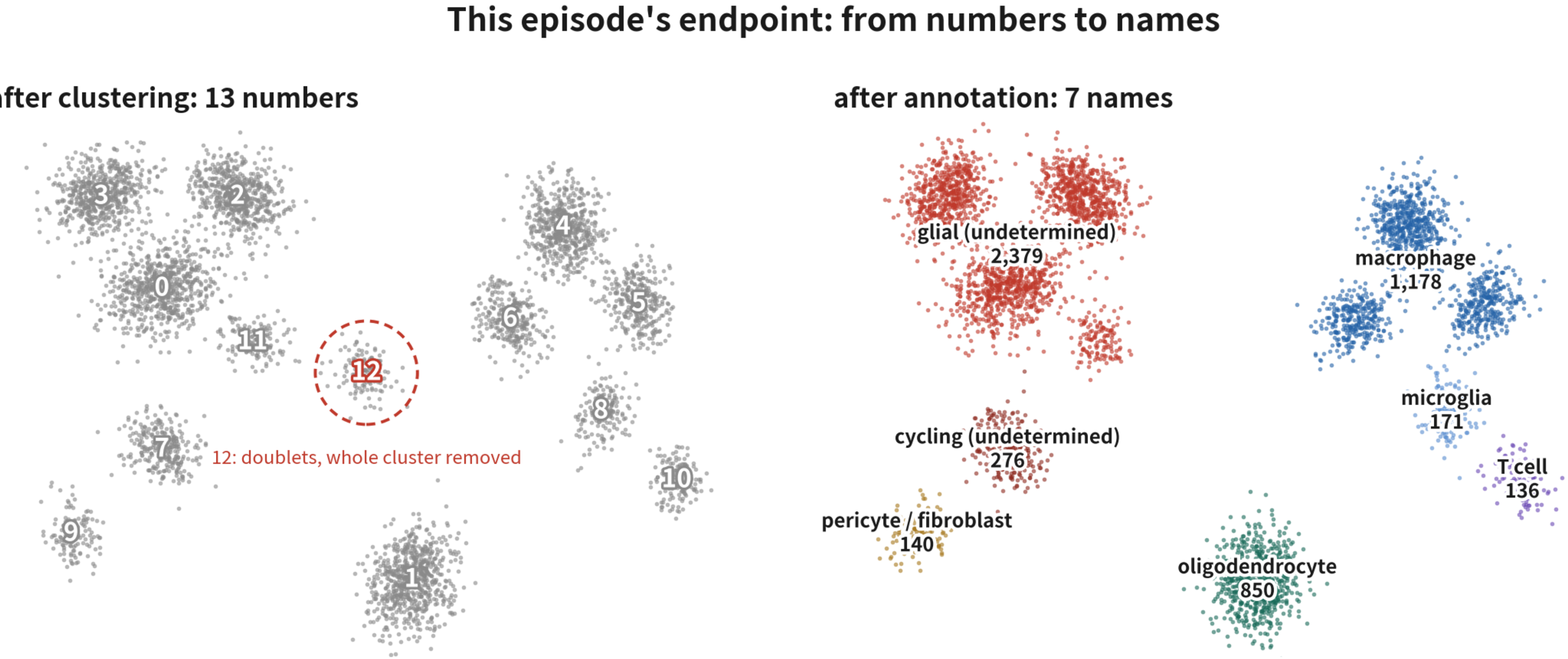
Hands-on I: the single-sample standard pipeline from raw matrix to cell annotation

10x's public GBM 5k, 5,604 cells. From loading to named clusters and state scores for malignant cells.

About 55 min | Prerequisites: Q1 | Scripts: R/00–03 | Deeper: B1, B5–B10

https://github.com/Charlene717/scRNA-seq-tutorial-Qseries

The end result of this episode



13 clusters is not 13 cell types: four are glial, three are macrophage, and one is a doublet cluster removed whole. Numbers reshuffle with any parameter; only names go into a paper

13 numbers, 7 names: four clusters are glial, three are macrophage, one doublet cluster is removed whole. Malignant cells are labelled 'glial (undetermined)', not declared tumour

THE RULE FOR THIS EPISODE

Every code block answers two questions: what this line does, and why the parameter is set that way.

Full code lives in three practice scripts (R/01–03), each ending with exercises.

‘Deeper: Bx’ points to the Series B episode that goes all the way.

Episode goals

1. Load GBM 5k into Seurat v5, set three QC cutoffs from this dataset‘s distributions, flag doublets

2. Run normalization → HVG → scaling → PCA → clustering → UMAP with three sanity checks (PC1, nPC, per-cluster QC)

3. Draw the four major groups with gate genes, name clusters with markers + SingleR, score malignant states with Neftel gene sets

01

Environment and data

Script 00: install, download, set up the project

Practice data: a real tumour sample (GBM 5k)

Practice data: Human Glioblastoma Multiforme 5k

source	10x Genomics public dataset (Chromium 3' v3)
tissue	dissociated human glioblastoma tumour
cells	5,604 (called by Cell Ranger 4.0.0)
files	filtered_feature_bc_matrix.tar.gz (≈ 30 MB)
why	A real tumour: malignant, immune, vascular and normal brain cells; runs on a laptop in 5 min

What it cannot answer

- ✕ One patient only: no patient-level inference (n=1)
- ✕ No matched normal: malignant calling relies on internal references (immune, oligodendrocytes)
- ✕ No clinical data: cannot link to response or survival
- → Q3 switches to a 4-patient dataset to fill these gaps

Read the right side first: what a dataset can and cannot answer matters more than its cell count

Install, pin the version, set the seed

```r
install.packages(c("Seurat", "dplyr", "ggplot2", "clustree"))
BiocManager::install(c("SingleR", "celldex", "scDblFinder"))
remotes::install_github("chris-mcginnis-ucsf/DoubletFinder")   # 只在 GitHub
library(Seurat); library(dplyr); library(ggplot2)
stopifnot(packageVersion("Seurat") >= "5.0.0")   # v4 教學碼在 v5 會炸
set.seed(1234)                                    # 全系列固定
for (d in c("data", "R", "output")) dir.create(d, showWarnings = FALSE)
```

No setwd(), no absolute paths: open an RStudio Project and the root is the working directory | Script 00 | Deeper: B1

Download, extract, load

```
url <- paste0("https://cf.10xgenomics.com/samples/cell-exp/4.0.0/",
              "Parent_SC3v3_Human_Glioblastoma/",
              "Parent_SC3v3_Human_Glioblastoma_filtered_",
              "feature_bc_matrix.tar.gz")
download.file(url, "data/gbm5k.tar.gz", mode = "wb")   # 約 30 MB
untar("data/gbm5k.tar.gz", exdir = "data/gbm5k")

mtx.dir <- "data/gbm5k/filtered_feature_bc_matrix"
gbm.data <- Read10X(data.dir = mtx.dir)               # 吃資料夾
gbm <- CreateSeuratObject(counts = gbm.data, project = "gbm5k",
                          min.cells = 3, min.features = 200)
gbm
```

```
An object of class Seurat
~25,000 features across ~5,500 samples within 1 assay
```

min.cells / min.features are a coarse screen; real cutoffs come from distributions next. Use the numbers you actually get | Script 01 §1

First checks after loading: dimensions, sparsity, gene names

```r
dim(gbm)                                         # 基因 × 細胞
cnt <- LayerData(gbm, layer = "counts")
1 - Matrix::nnzero(cnt) / prod(dim(cnt))   # 稀疏度：> 0.9 才正常
summary(Matrix::colSums(cnt))              # 每顆細胞的 UMI 總數
grep("^MT-", rownames(gbm), value = TRUE)  # 人 MT-、鼠 mt-
head(grep("^RP[LS]", rownames(gbm), value = TRUE))   # 核糖體
cnt[c("PTPRC", "SOX2", "MBP"), 1:5]        # 三個譜系標誌基因
```

```
[1] 25xxx  55xx
[1] 0.93
   Min. 1st Qu. Median   Mean 3rd Qu.   Max.
    5xx    2xxx   4xxx   6xxx   8xxx  6xxxx
 [1] "MT-ND1" "MT-ND2" "MT-CO1" ...  (13 genes)
```

Thirty seconds that prevent three accidents: wrong species (MT- vs mt-), loading the raw matrix (hundreds of thousands of columns), Ensembl IDs instead of symbols | Script 01 §1

raw, filtered, and ambient RNA

Cell Ranger hands you two matrices; we load filtered, but raw has one more use

raw matrix

raw_feature_bc_matrix: every Barcode (hundreds of thousands), mostly empty droplets. The RNA in empty droplets is a sample of the background 'soup'

filtered matrix

filtered_feature_bc_matrix: Barcodes Cell Ranger calls cells via EmptyDrops (5,604 here). This is what we load; doublets and damaged cells are still inside

Ambient RNA

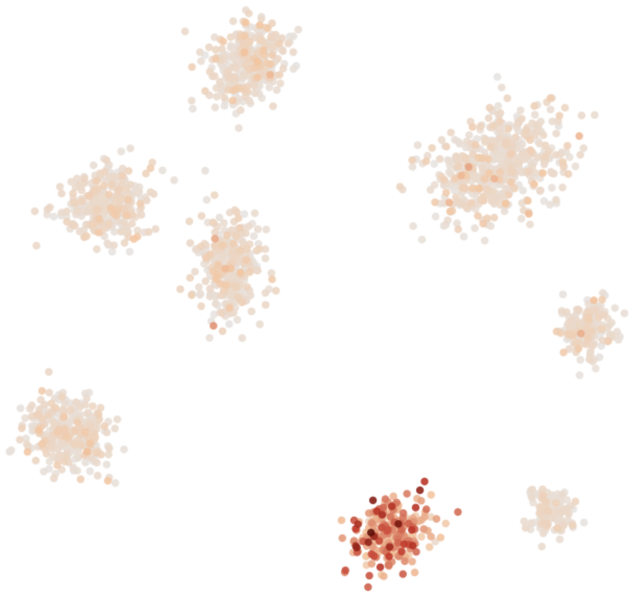
mRNA from lysed cells drifts into every droplet: each cell carries a little of everyone else. Worst in necrotic, slowly dissociated tumours

Ambient RNA: how to spot it, how to remove it

Ambient RNA: every cell in the matrix carries a little 'soup'

schematic / simulated

before: MBP



every cell carries a little MBP—not because oligodendrocytes are everywhere, but because mRNA from lysed cells drifts into every droplet

after SoupX: MBP

background estimated and subtracted; the marker lights only where it should

What SoupX does

① estimate the soup profile from empty droplets (low-UMI Barcodes in the raw matrix)

② estimate each cell's contamination fraction ρ from genes a cell type cannot express (e.g. MBP outside oligodendrocytes)
③ subtract ρ×soup from every cell, return corrected integer counts

When it is mandatory: necrotic, slowly dissociated tumours; ρ> 10%; when gate genes (PTPRC, MBP, HBB) show a little everywhere

A gate gene showing a little everywhere → estimate contamination first; above ρ ≈ 10%, run SoupX (or CellBender) before QC

SoupX: estimating and removing ambient RNA (optional, script 01 §0)

```
library(SoupX)
sc  <- load10X("data/gbm5k_outs/")           # 需 raw + filtered 同在
sc  <- setClusters(sc, gbm$seurat_clusters)  # 先粗分群
sc  <- autoEstCont(sc)                       # 估污染比例 rho
sc$fit$rhoEst                                # 例：0.06 → 可做可不做
adj <- adjustCounts(sc, roundToInt = TRUE)   # 校正後整數 counts
gbm <- CreateSeuratObject(adj, project = "gbm5k",
                          min.cells = 3, min.features = 200)
```

Needs the raw matrix, so datasets that ship filtered-only cannot do this; always keep raw for your own data.
Skip below ρ ≈ 5% | Deeper: B5

02

QC: three cutoffs set from the data

Look first, cut second; save the raw object before filtering

The three QC metrics: inspect the distributions before filtering

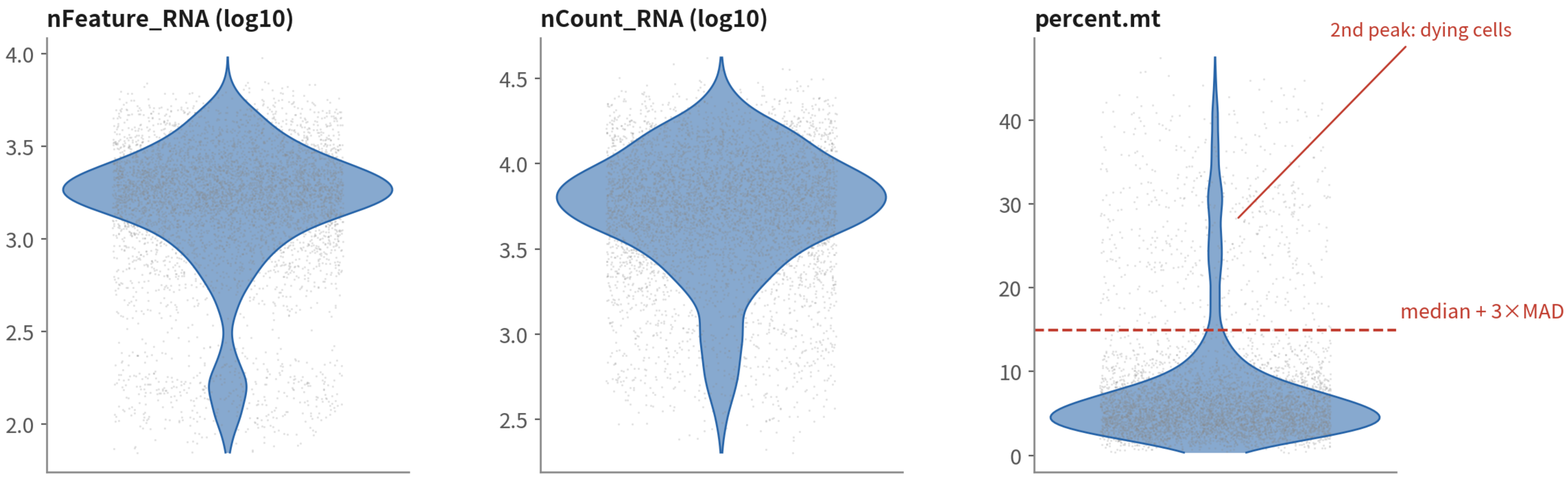
```
gbm[["percent.mt"]] <- PercentageFeatureSet(gbm, pattern = "^MT-")
saveRDS(gbm, "output/rds/01_gbm_raw.rds")     # 過濾前先存：截斷之後回不去

VlnPlot(gbm, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"),
        ncol = 3, pt.size = 0.1)
FeatureScatter(gbm, "nCount_RNA", "percent.mt")
FeatureScatter(gbm, "nCount_RNA", "nFeature_RNA")
```

nCount catches empties and doublets, nFeature debris, percent.mt dying cells | Script 01 §2 | Deeper: B5

QC distributions: a tumour sample as the example

The three QC metrics (typical tumour sample)

schematic / simulated



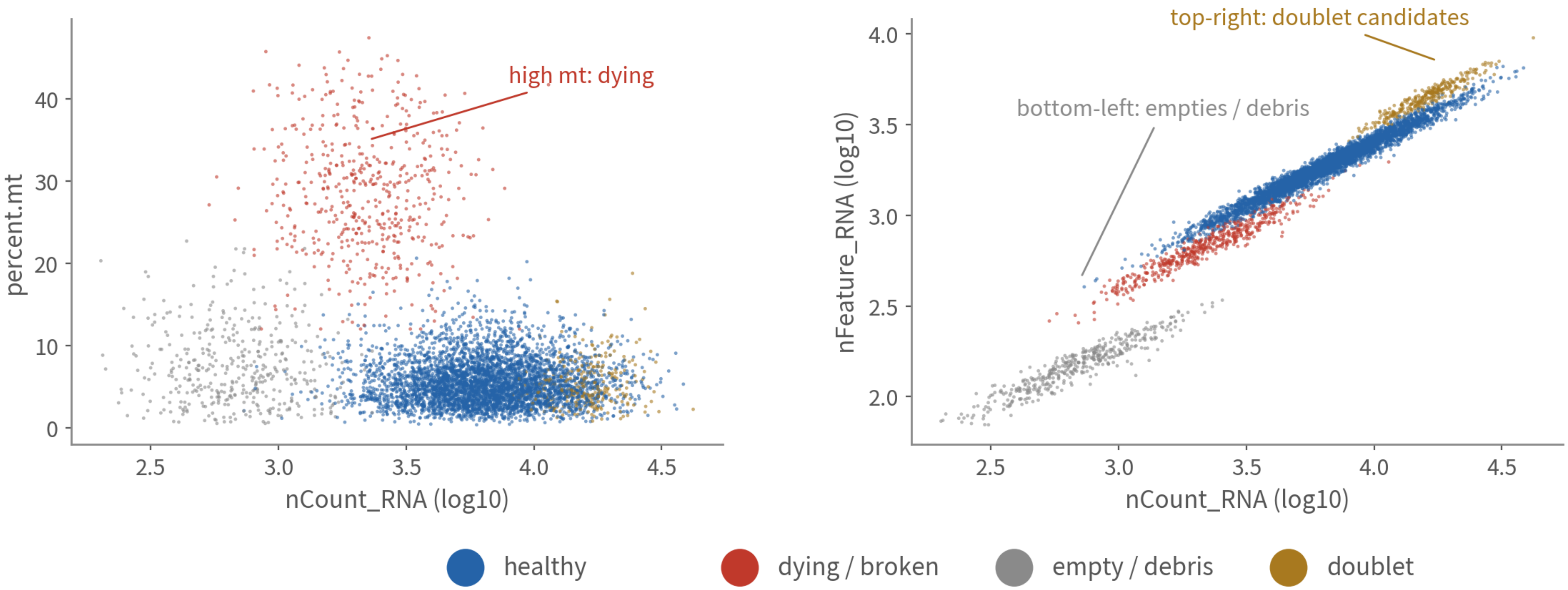
Tumour percent.mt runs higher than PBMC (malignant cells are metabolically active) and is often bimodal. A copied 5% cut removes a large slice of live cells

percent.mt runs higher than PBMC and is often bimodal. The second mode is dying cells; the line goes between the modes

QC scatter plots: where the three failure modes sit

Two scatters read together: each kind of junk owns a corner

schematic / simulated



One metric misleads; read them in pairs. The top-right doublet candidates get confirmed with a dedicated tool shortly

Filtering with MAD-based adaptive cutoffs

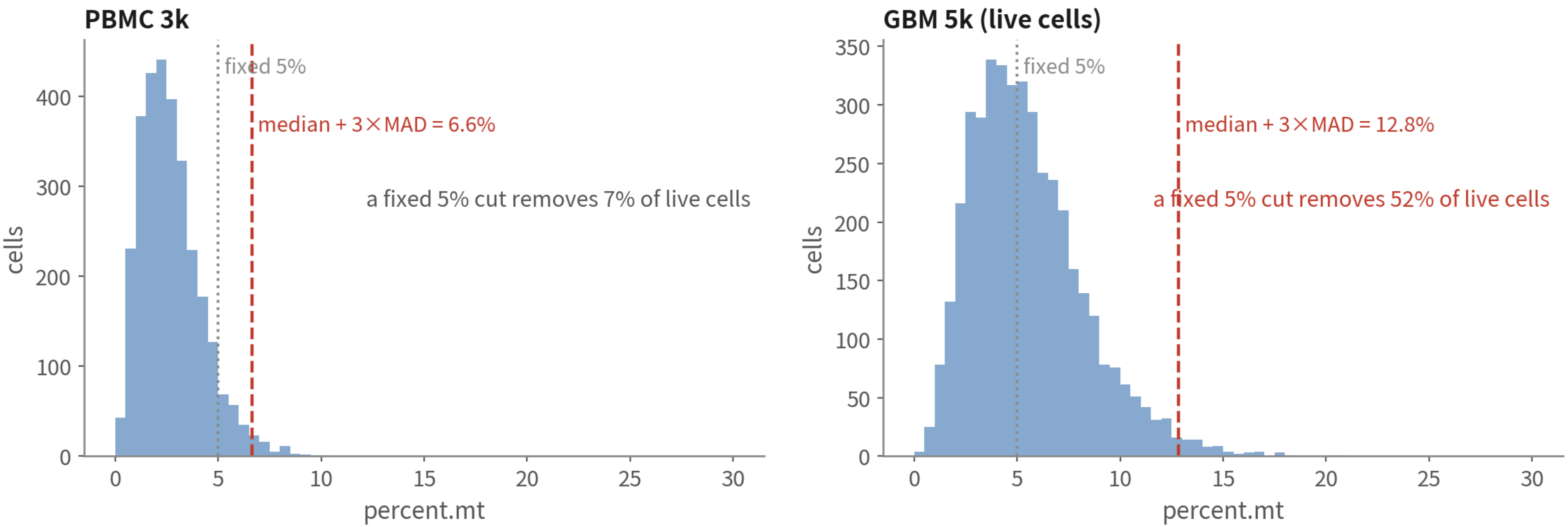
```
mad.hi <- function(x, k = 3) median(x) + k * mad(x)   # mad() 已含 1.4826
mad.lo <- function(x, k = 3) median(x) - k * mad(x)

mt.hi  <- mad.hi(gbm$percent.mt)                    # 例：≈ 12.8（依你的資料）
nf.lo  <- 10^mad.lo(log10(gbm$nFeature_RNA))    # 在 log 尺度算下限
nc.hi  <- 10^mad.hi(log10(gbm$nCount_RNA))      # 在 log 尺度算上限
c(mt.hi = mt.hi, nf.lo = nf.lo, nc.hi = nc.hi)

gbm <- subset(gbm, subset = percent.mt < mt.hi &
                            nFeature_RNA > nf.lo & nCount_RNA < nc.hi)
gbm                                                  # 記下濾掉幾顆
```

nCount / nFeature are log-normal, so compute MAD on log10; percent.mt as is | Script 01 §3

One rule, two datasets, two different lines

Same rule, two datasets, two different lines — that is why it beats copying 5%

schematic / simulated



A copied 5% cut on GBM removes half the live cells — mostly the metabolically active malignant ones

COPY-READY METHODS SENTENCE

“Cells were retained with: percent.mt < median + 3×MAD (12.8%); nFeature and nCount within median ± 3×MAD on log10 scale; N of 5,604 cells kept."

Fill N and the three numbers with your own results.

Every cutoff has a number and a reason.

Doublets: flag with DoubletFinder, do not delete yet

```
library(DoubletFinder)
# DoubletFinder 在 PC 空間找鄰居 → 先做一份暫時物件（正式分群在 02）
tmp <- NormalizeData(gbm) |> FindVariableFeatures(nfeatures = 2000) |> ScaleData() |>
       RunPCA(npcs = 30) |> FindNeighbors(dims = 1:20) |> FindClusters(resolution = 0.5)
# ① 掃 pK：唯一要調的參數
bcmvn  <- find.pK(summarizeSweep(paramSweep(tmp, PCs = 1:20, sct = FALSE), GT = FALSE))
pK.sel <- as.numeric(as.character(bcmvn$pK[which.max(bcmvn$BCmetric)]))
# ② 期望 doublet 數要「你自己給」：10x 每千顆約 0.8%，再扣掉同型 doublet
nExp <- round(0.008 * ncol(tmp) / 1000 * ncol(tmp))
nExp <- round(nExp * (1 - modelHomotypic(tmp$seurat_clusters)))
tmp  <- doubletFinder(tmp, PCs = 1:20, pN = 0.25, pK = pK.sel, nExp = nExp, sct = FALSE)
pann <- grep("^pANN_",    colnames(tmp@meta.data))[1]   # 欄名帶參數，用 grep 取
dfcl <- grep("^DF.class", colnames(tmp@meta.data))[1]
gbm$dbl.score <- tmp@meta.data[[pann]]
gbm$dbl.class <- tolower(tmp@meta.data[[dfcl]])
table(gbm$dbl.class)
```

Expected rate 4.2% (5,261 items) → nExp = 221 → After deducting the number of doublets of the same type, it equals N.
The number of doublets in table() is equal to N — this is what you provided, not what the tool found itself.

You supply the rate and the flagged count equals nExp exactly — that number belongs in your methods | Script 01 §4 | Deeper: B5

What DoubletFinder does

Doublets: the junk QC metrics miss — use a dedicated tool

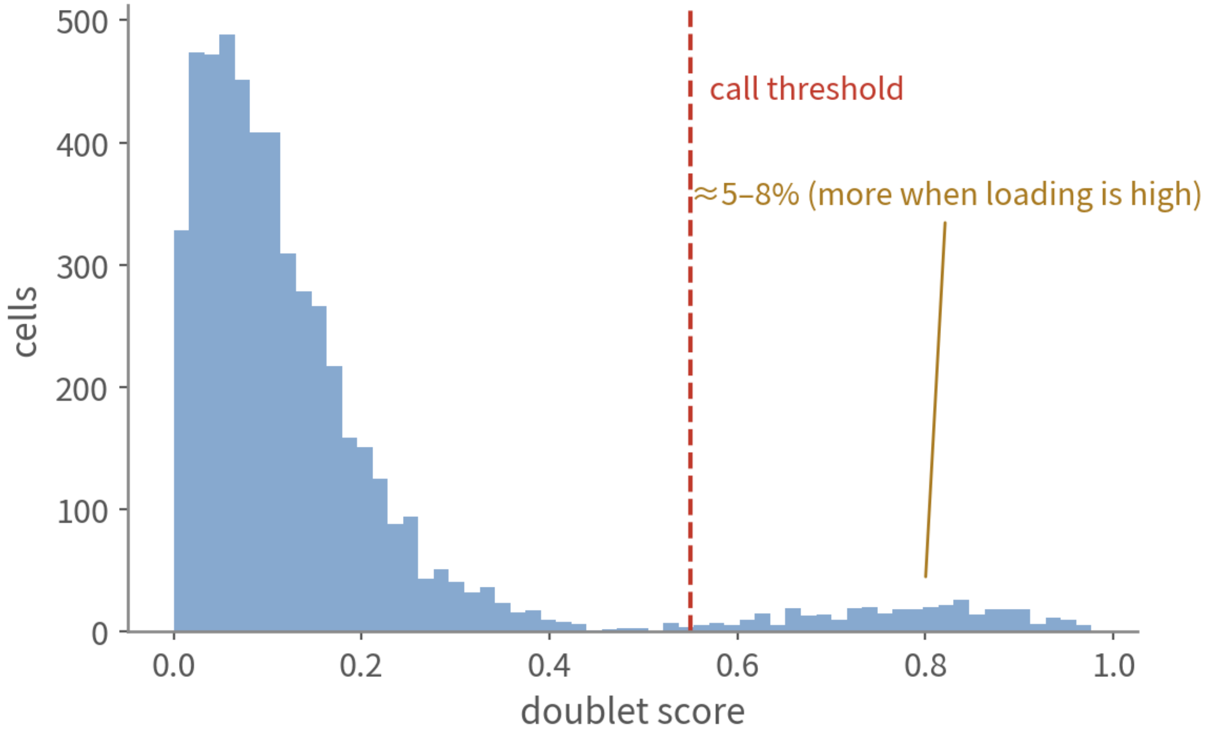
schematic / simulated

How DoubletFinder thinks

① pick two cells at random, add their profiles→artificial doublets

② drop them into PC space and see which real cells sit next to them

③ cells surrounded by artificial doublets get a high score



Tumour doublet risk is high: a malignant cell stuck to a macrophage looks like a 'tumour cell expressing immune genes'

Choosing a doublet tool, and what rate to expect

The more cells loaded, the more doublets: the 10x rule of thumb is ≈ 0.8% per 1,000 cells

Tool	Idea	Strengths	Watch out
DoubletFinder (this course)	Artificial doublets + pANN fraction; the top nExp cells are called	Published in Cell Systems; top for accuracy in the Xi & Li 2021 benchmark	You supply both the expected rate and pK; the pK sweep is slow
scDblFinder (cross-check)	Artificial doublets + kNN classifier; rate and threshold estimated automatically	One line, Bioconductor-maintained, native multi-sample	Published in F1000Research; not included in that benchmark
Scrublet (Python)	Same idea, built into scanpy	The default in Python pipelines	Nearly blind to homotypic doublets
Hashing / demuxlet	Wet-lab: antibody tags or SNPs to split samples	The only way to catch homotypic doublets; free when pooling patients	Decided at design time not afterwards

03

Normalize, HVG, scale, PCA

Script 02: four lines of code, three sanity checks

Three problems, three steps

Problem 1: depth differs

Malignant cells carry more RNA than T cells: totals differ 5–10×. Comparing raw counts compares depth

`NormalizeData`

divide by total × 1e4, log1p

Problem 2: too many genes

Most of 36k genes do not vary between cells—just noise

`FindVariableFeatures`

keep 2,000 genes varying beyond technical expectation

Problem 3: unequal scale

MALAT1 in thousands, TFs in single digits. Unscaled, PCA is hijacked by loud genes

`ScaleData`

z-score per gene, clip ±10

Every later step consumes this output: PCA runs on the scale.data of the 2,000 HVGs

A malignant cell often carries ten times a T cell's RNA — step one removes exactly that difference

Four preprocessing steps, plus cell-cycle scores

```
gbm <- NormalizeData(gbm)                          # CP10K + log1p：除掉深度
gbm <- FindVariableFeatures(gbm, nfeatures = 2000)   # 挑 2,000 個 HVG
head(VariableFeatures(gbm), 10)                    # 先看一眼名單

gbm <- CellCycleScoring(gbm, s.features = cc.genes.updated.2019$s.genes,
                        g2m.features = cc.genes.updated.2019$g2m.genes)
gbm <- ScaleData(gbm)                              # z-score、clip ±10（只做 HVG）
gbm <- RunPCA(gbm, npcs = 50)                      # 50 個 PC 先算存著
```

Score the cycle now so you can see whether cycling cells form their own cluster; we do not regress by default | Script 02 §1 | Deeper: B6

LogNormalize or SCTransform?

Both are valid; this course uses the former because every step is visible and it plays well with pseudobulk and inferCNV

LogNormalize (this course)

CP10K + log1p; HVG via vst

- Simple, fast, interpretable; the data layer feeds FeaturePlot / DotPlot directly
- The counts layer stays untouched — pseudobulk, inferCNV and SoupX all need it
- With extreme depth differences (malignant vs T cells) a little depth
  effect remains — the PC1 check catches it

SCTransform

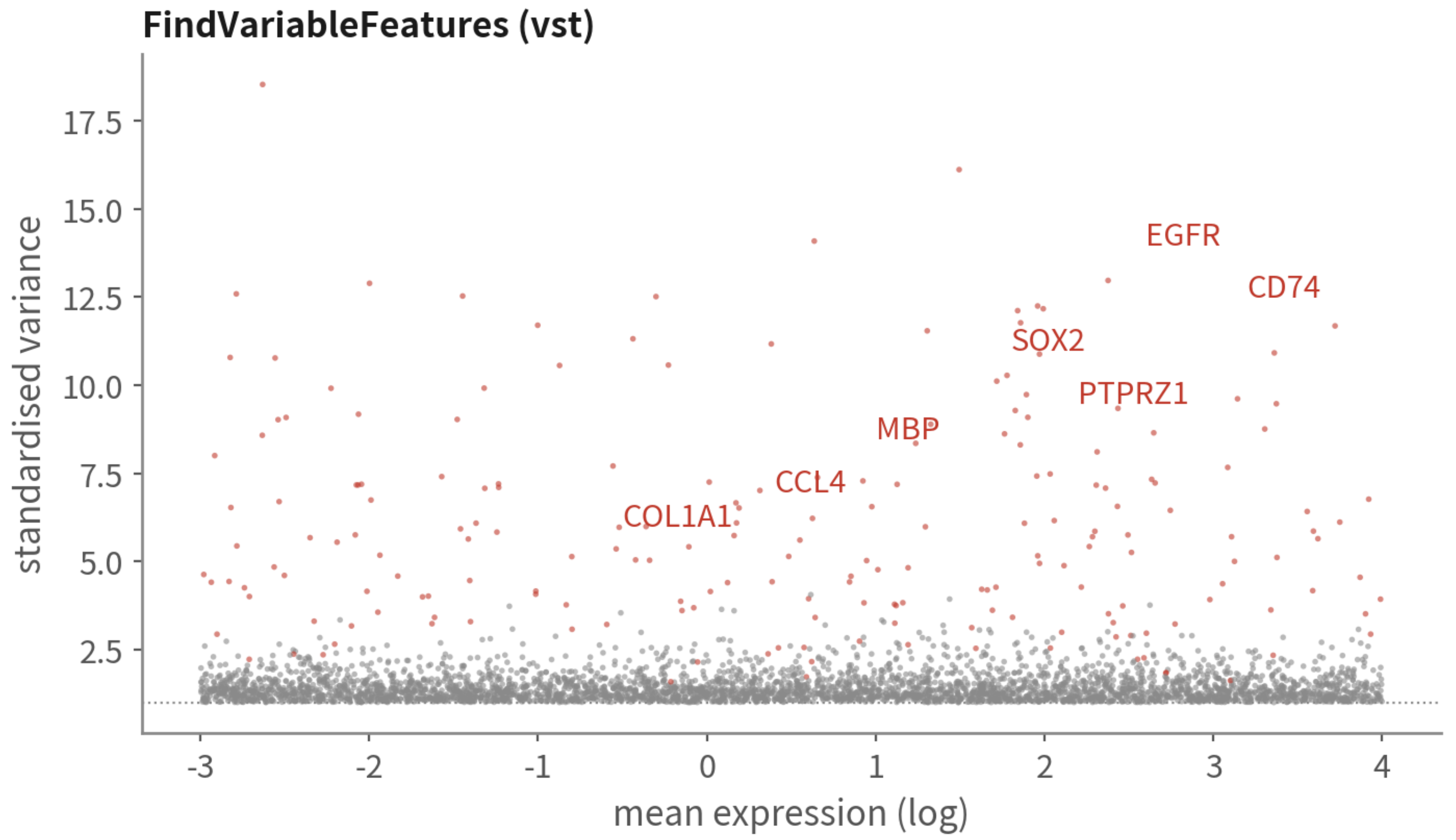
Negative-binomial regression; depth removed more cleanly

- Steadier HVGs, often cleaner clusters; v2 handles wide depth ranges well
- Adds an SCT assay; run PrepSCTFindMarkers before FindMarkers; run it per sample before integrating
- DE, CNV and pseudobulk still come from RNA counts — never from SCT values

The HVG list carries biology — but the ranking misleads

HVG: 2,000 genes with something to say, out of 36,000

schematic / simulated



Who tops the GBM HVG list

- malignant-vs-normal genes (EGFR, SOX2, PTPRZ1)
- immune genes (CD74, HLA, CCL4)
- oligodendrocyte (MBP, PLP1), vascular (VWF)
- The list itself is biology— look at it before moving on

The top 20 here are almost all stromal genes, from a 140-cell fibroblast cluster. HVG ranks variance, not importance | Script 02 §1

Sanity check 1: PC1 is biology, not depth

```
print(gbm[["pca"]], dims = 1:2, nfeatures = 6)
cor(Embeddings(gbm, "pca")[, 1], gbm$nCount_RNA)   # < 0.3 放心；0.3–0.5 看載荷；> 0.5 要查
```

```
PC_ 1
Positive:  PMP2, CALD1, PTN, GAP43, ITM2C, SEC61G   ← glial / malignant
Negative:  LAPTM5, TYROBP, SRGN, FCER1G, CYBA, AIF1  ← immune
[1] 0.47
```

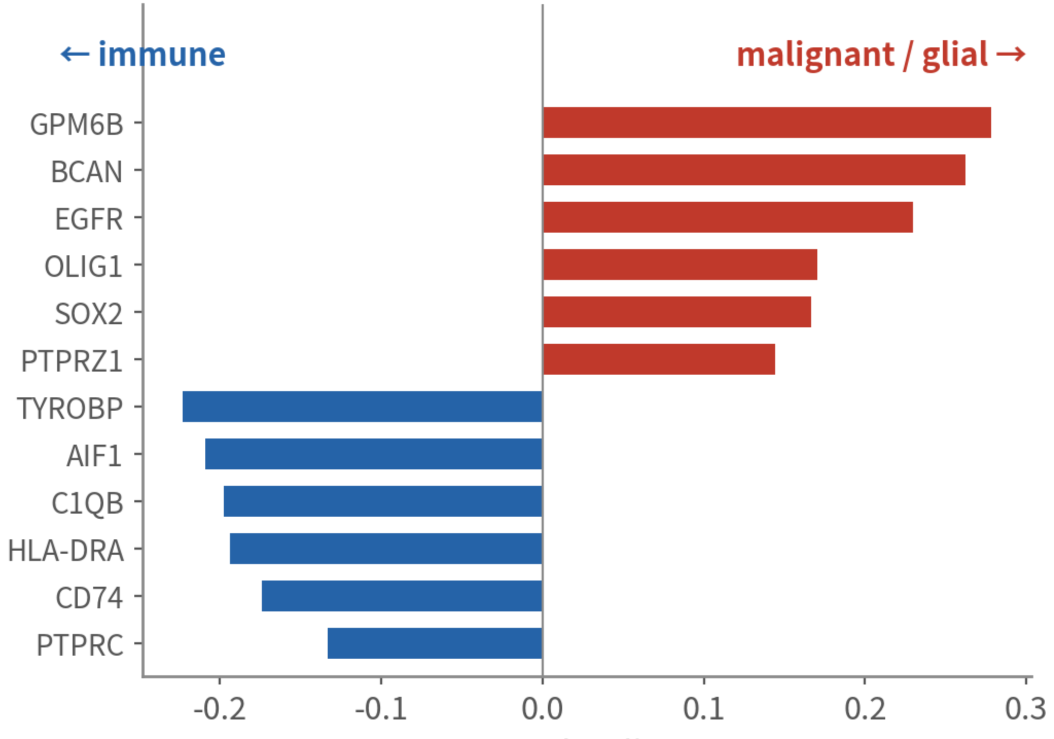
In GBM PC1 is almost always immune vs glial. 0.47 sits in the usual tumour band — judge by the loadings, not the coefficient | Script 02 §2 | Deeper: B7

PC1 and the elbow

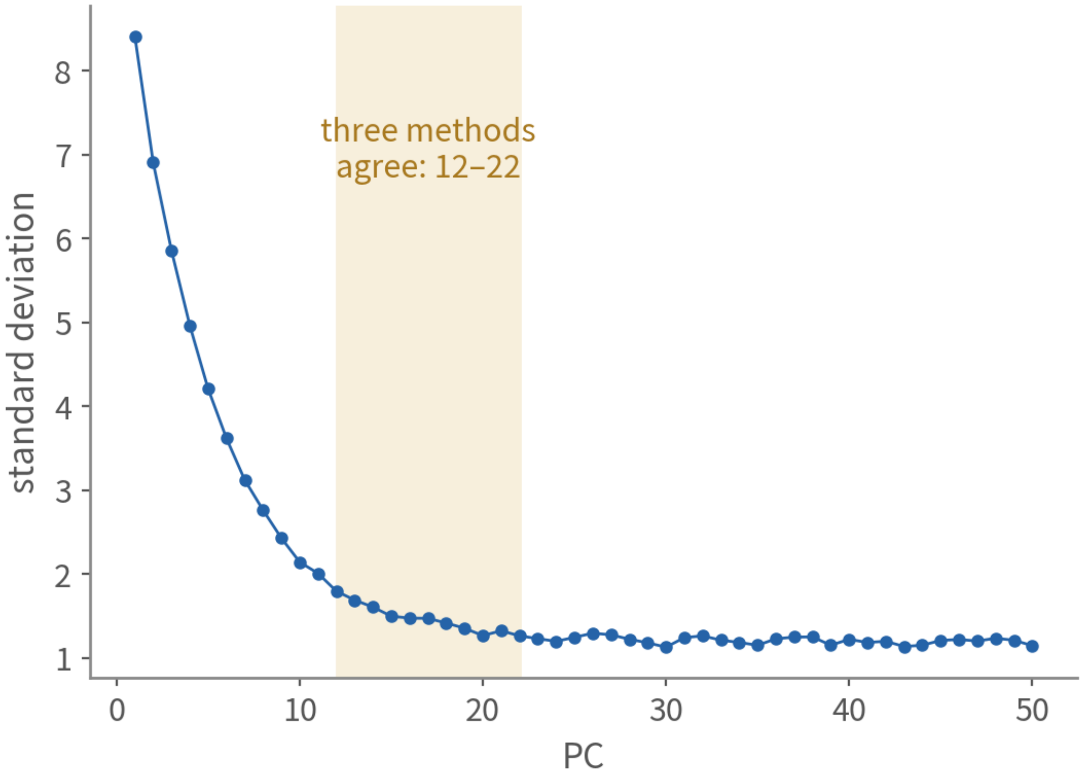
PCA: check PC1 is biology first, then choose how many

schematic / simulated

PC1 = the immune vs malignant/glial axis



ElbowPlot: tumours elbow later than PBMC



Tumours are structured (many types+malignant states+patient effects):
use dims 20–30, more than PBMC's 10. More is safer than fewer

Tumours are structured, so the elbow comes later than PBMC. Use dims 20–30; more is safer than fewer

Sanity check 2: how many PCs

```
ElbowPlot(gbm, ndims = 50)
pct  <- gbm[["pca"]]@stdev / sum(gbm[["pca"]]@stdev) * 100
cumu <- cumsum(pct)
which(cumu > 90 & pct < 5)[1]        # 累積 90% 且單 PC < 5%
npc <- 25                            # GBM：三法交集約 20–30，取 25
```

Turn the elbow into a number you can write in the methods: too few PCs merges subtypes away, too many only adds a little noise | Script 02 §2

Regress out the cell cycle? Check two things first

Cycling cells in tumours are biology; do not regress by default, unless the cycle hijacks the clustering

Do not regress (default)

Cycling cells form their own cluster; other clusters still split by type

- Check DimPlot(group.by = “Phase”): only a small S / G2M patch
- Check: no MKI67 / TOP2A among PC1–PC5 loadings
- Handling: annotate that cluster as 'cycling X', not a new type(Pitfall 3)

Regress (vars.to.regress)

When every type is cut in two by the cycle

- ScaleData(vars.to.regress = c("S.Score", "G2M.Score"))
- Cost: the proliferation signal vanishes — often exactly what tumour studies want
- Same logic for percent.mt: regress only when it dominates a PC

04

Clustering and UMAP

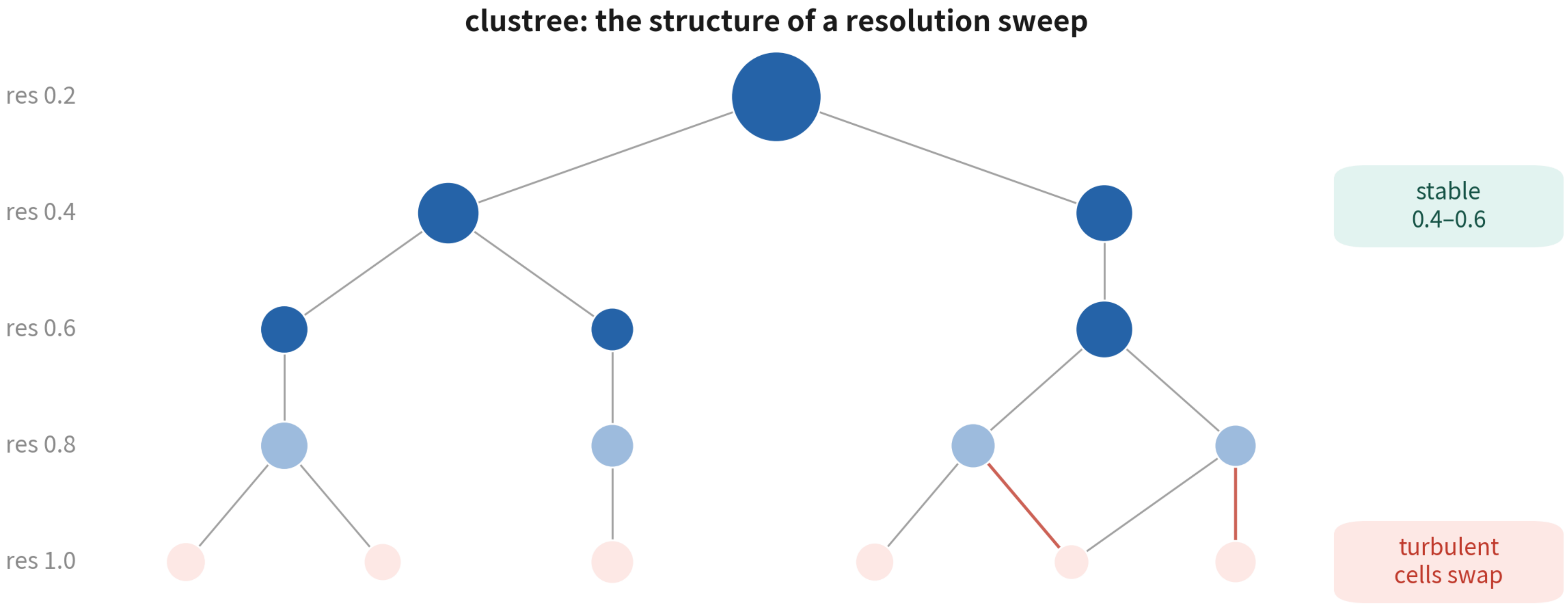
Cluster in PC space; sweep resolution; QC again after clustering

Neighbour graph → communities → projection → sweep

```
gbm <- FindNeighbors(gbm, dims = 1:npc)              # PC 空間建 KNN/SNN 圖
gbm <- FindClusters(gbm, resolution = 0.5)           # 起點，不是答案
gbm <- RunUMAP(gbm, dims = 1:npc)                    # 同一組 dims

for (r in seq(0.2, 1.2, by = 0.2))
  gbm <- FindClusters(gbm, resolution = r, verbose = FALSE)
library(clustree); clustree(gbm, prefix = "RNA_snn_res.")
Idents(gbm) <- "RNA_snn_res.0.5"                      # 掃完指回定案欄位
DimPlot(gbm, label = TRUE)
```

After the sweep the active identity is the last one run — always point it back | Script 02 §3 |
Deeper: B8, B9

clustree: find the stable band



Find the band where several layers agree; step back where arrows cross and cells swap. Every extra split needs marker support

Tumour data separates immune / glial / oligo at low resolution; further cuts are malignant states and immune subsets — each needs marker support

Are the clusters stable? Change the seed, change k, watch the sizes

```
# 1. 同一 resolution 換 seed：ARI 接近 1 才算穩
gbm <- FindClusters(gbm, resolution = 0.5, random.seed = 42, cluster.name = "seed42")
mclust::adjustedRandIndex(gbm$RNA_snn_res.0.5, gbm$seed42)   # 例：0.9x

# 2. 換 k.param（預設 20）：群的骨架不該因此改變
gbm <- FindNeighbors(gbm, dims = 1:20, k.param = 30)
gbm <- FindClusters(gbm, resolution = 0.5, cluster.name = "k30")
table(gbm$RNA_snn_res.0.5, gbm$k30)          # 對角線清楚 = 穩

# 3. 群大小：小於 30 顆的群先存疑
sort(table(gbm$RNA_snn_res.0.5))
```

Ten seconds for three checks. ARI < 0.8, or a cluster that vanishes when k changes → an unstable cut; let markers decide whether it stays | Script 02 §3

Sanity check 3: per-cluster QC and doublet review

```
gbm@meta.data |>
  group_by(cluster = RNA_snn_res.0.5) |>
  summarise(n = n(), mt = median(percent.mt), nF = median(nFeature_RNA),
            dbl = mean(dbl.class == "doublet"), cyc = mean(Phase != "G1")) |>
  arrange(desc(dbl))
```

```
cluster     n    mt    nF   dbl   cyc
12        131  5.1  6221  1.00  0.24   ← all doublets, highest nF → drop the cluster
 9        140  3.6  5370  0.34  0.20
 7        276  4.4  5625  0.11  1.00   ← cyc = 1: a proliferative STATE, not a type
 2        609  3.5  5166  0.09  0.13
...
```

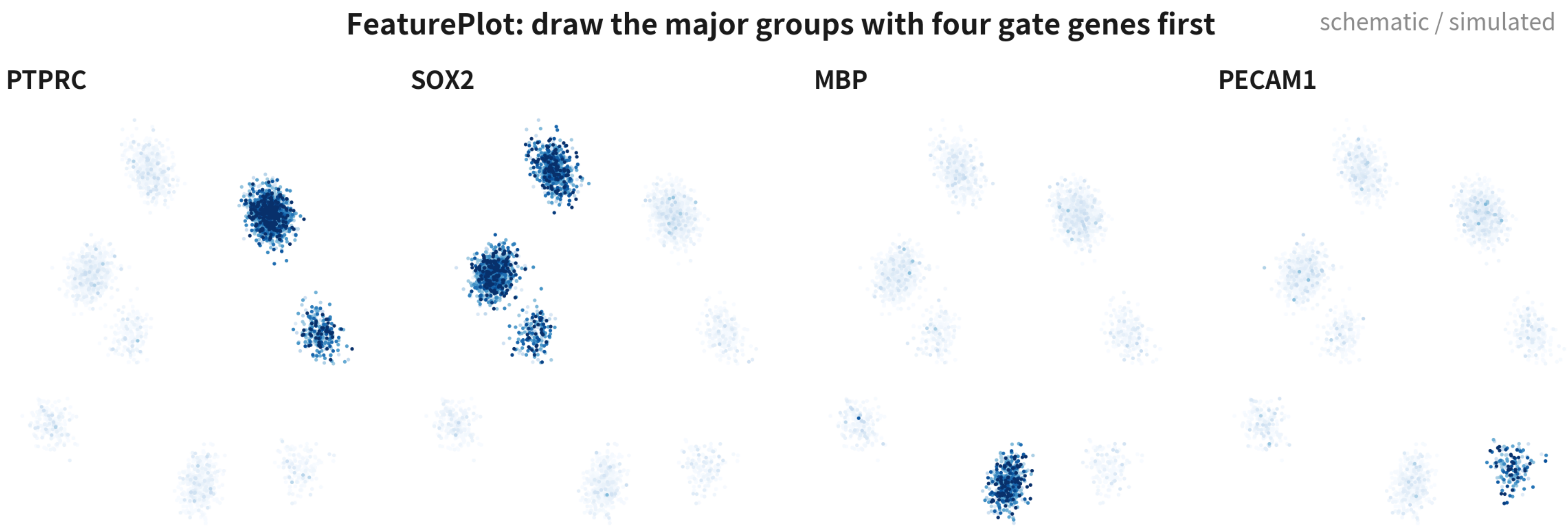
A cluster that is mostly doublets and sits between two big groups → remove and record; a cluster with runaway mt → back to QC | Script 02 §4

05

Annotation: four major groups, then subtypes

Script 03: gate genes → markers → SingleR → state scores

Draw the major groups with four gate genes first



Immune (PTPRC), glial/malignant (SOX2), oligodendrocyte (MBP), vascular (PECAM1)—settle the four major groups, then the subtypes

Immune (PTPRC), glial/malignant (SOX2), oligodendrocyte (MBP), vascular (PECAM1). Settle the four, then subtypes

Gate genes and the marker panel

```r
FeaturePlot(gbm, features = c("PTPRC", "SOX2", "MBP", "PECAM1"), ncol = 4)

panel <- c("PTPRC", "CD68", "CD163", "P2RY12", "TMEM119", "CD3E",   # 免疫
           "SOX2", "OLIG2", "PTPRZ1", "EGFR",                      # 膠質 / 惡性
           "MBP", "PLP1", "PECAM1", "VWF", "PDGFRB", "RGS5",  # 寡樹突 / 血管 / 周細胞
           "MKI67", "TOP2A")                                       # 週期
DotPlot(gbm, features = panel) + RotatedAxis()
```

Blocks along the diagonal mean good clusters; PTPRC lighting three immune types is expected | Script 03 §1 | Deeper: B10

A marker panel for the GBM microenvironment

A marker panel for the GBM tumour microenvironment

population	canonical markers	one line
immune group	`PTPRC (CD45)`	gate for all immune cells
macrophage	`CD68, CD163, C1QB, AIF1`	tumour-associated macrophages, the most abundant immune cells in GBM
microglia	`P2RY12, TMEM119, CX3CR1`	brain-resident macrophages; distinguish from blood-derived TAM
T cell	`CD3E, CD3D, CD2 (+CD8A / IL7R)`	GBM is immune-cold; T cells are usually scarce
malignant / glial	`SOX2, OLIG2, PTPRZ1, EGFR`	markers only say 'glial lineage'; malignancy needs CNV (Q3)
oligodendrocyte	`MBP, PLP1, MOG`	invaded normal white matter; a good inferCNV reference
endothelial	`PECAM1, VWF, CLDN5`	prominent vascular proliferation in GBM
pericyte	`PDGFRB, RGS5, ACTA2`	perivascular stromal cells
cycling	`MKI67, TOP2A`	a state, not a type; overlays malignant cells

Microglia vs blood-derived macrophages, and cycling state vs cell type — the two pairs most often confused in tumour annotation

Evidence 1: the FindAllMarkers table

```
markers <- FindAllMarkers(gbm, only.pos = TRUE,
                          min.pct = 0.25, logfc.threshold = 0.5)
markers |> group_by(cluster) |> slice_max(avg_log2FC, n = 5) |>
  select(cluster, gene, avg_log2FC, pct.1, pct.2) |> print(n = 50)
```

```
cluster gene   avg_log2FC pct.1 pct.2
2       C1QB   high       ~0.95  ~0.05   ← macrophage
4       MBP    high       ~0.98  ~0.08   ← oligodendrocyte
0       PTPRZ1 high       ~0.9   ~0.2    ← glial (malignancy undetermined)
```

Read in order: avg_log2FC → pct.1 / pct.2 → p. pct.2 near 0 is the clean marker | Script 03 §2

Filtering the marker table: four columns, four questions

Look at p last — with 5,000 cells nearly everything is significant

Column	It answers	A good marker	Trap
avg_log2FC	How much higher than the rest	> 1 (over two-fold)	Low genes get their FC squashed by the pseudocount
pct.1	Fraction expressing inside the cluster	> 0.5	A 'marker' at pct.1 = 0.3 describes part of the cluster — a hidden subset or doublets
pct.2	Fraction expressing elsewhere	< 0.1	Ambient RNA inflates pct.2; MBP at pct.2 = 0.3 does not mean oligodendrocytes everywhere
p_val_adj	Whether the difference is chance	< 0.05 (almost all are)	Cells in a cluster are not independent; p is overstated by orders of magnitude — use only as a sieve

Evidence 2: SingleR automated annotation

```
library(SingleR); library(celldex)
ref  <- celldex::HumanPrimaryCellAtlasData()
pred <- SingleR(test = GetAssayData(gbm, layer = "data"), ref = ref,
                labels = ref$label.main, clusters = gbm$seurat_clusters)
data.frame(cluster = rownames(pred), SingleR = pred$labels,
           delta = round(pred$delta.next, 2))
```

```
cluster SingleR           delta
0       Astrocyte          0.12  ← no GBM malignant in the reference — forced to the nearest
1       Astrocyte          0.08  ← actually oligodendrocyte (MAG/HAPLN2); HPCA has none either
4       Macrophage         0.18
9       Tissue_stem_cells  0.05  ← actually pericyte / fibroblast
10      T_cells            0.06  ← immune types are where it is genuinely accurate
```

Immune types are accurate; anything absent from the reference (malignant, oligodendrocyte) gets forced into the nearest normal type — a reminder, not an answer | Script 03 §3

Where automatic annotation stands (tumour data as the example)

Automatic annotation (SingleR /Azimuth) on tumour data

- ✓ Immune cells: accurate—references (HPCA, Blueprint) cover them well
- ✓ Oligodendrocytes, endothelium: accurate
- ✗ Malignant cells: no 'GBM malignant' in any reference—forced onto the nearest normal type (astrocyte, neural progenitor)
- → Verdict: use it as a second line of evidence; call malignancy separately

Cross-check table (example)

cl.	manual	SingleR	verdict
2	macrophage	Macrophage	✓
3	T cell	T_cells	✓
4	oligo	Oligodendrocyte	✓
0	malig. AC-like	Astrocyte	CNV decides
1	malig. OPC-like	Neural progenitor	CNV decides
5	endothelial	Endothelial	✓

Three-column cross-check: agreeing clusters pass; for the glial clusters both columns stop at lineage — this dataset cannot call malignancy

Annotation strategies: four roads, usually walked together

No single road is reliable alone; this course goes gate → markers → reference → scores. All four stop at lineage — malignancy needs a different class of evidence

Strategy	How	Strength	Weakness	Tools
Manual markers	Dotplot / FeaturePlot per cluster against literature markers	Interpretable; finds novel types	Slow, subjective, cluster-dependent	Seurat
Reference set	Correlate each cell / cluster with an annotated reference	Fast, reproducible; accurate for normal types	Types absent from the reference (tumour) get forced into the nearest	SingleR, scType, CellTypist
Gene-set scoring	Mean / AUC of a gene set as a score	Continuous, comparable suits states	High score ≠ type; thresholds are subjective	AddModuleScore, UCell, AUCell
Atlas mapping	Project cells onto a large atlas and inherit labels	Finest subsets, consistent labels	Needs a good atlas malignant cells cannot be mapped	Azimuth, scArches

Choosing an automatic annotation tool

Run all three on the same GBM data: normal types agree above 90%, and the disagreements are exactly where a human has to look

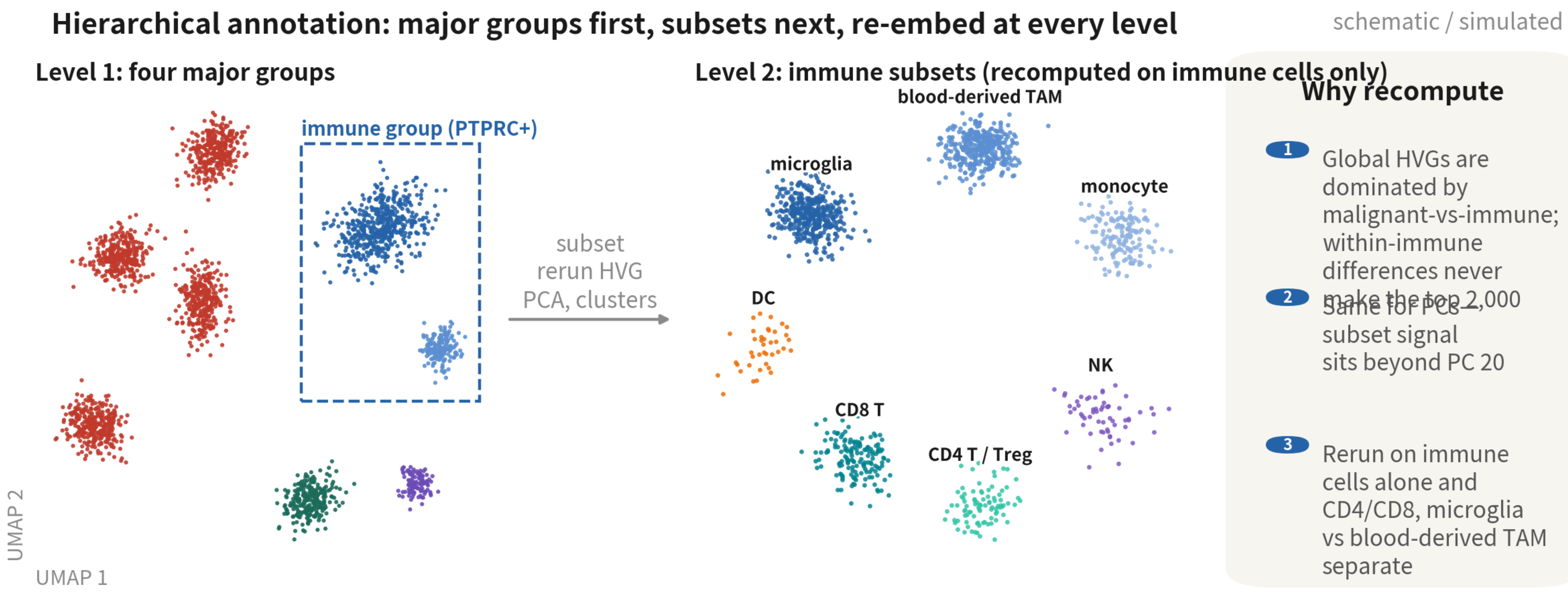
Tool	Idea	Reference	Unit	Best for / watch out
SingleR (this course)	Spearman correlation + fine-tuning	celldex (HPCA, Blueprint, Monaco immune)	Cell or cluster	Default for normal types; an immune reference splits T subsets
scType	Marker database + cluster-level scoring	Built-in human / mouse tissue markers	Cluster	Results in seconds; brain table has no tumour
CellTypist	Logistic-regression models	Immune atlas, multiple human tissues	Cell	Strongest on immune subsets (Python)
Azimuth	Anchor mapping onto a reference UMAP	PBMC, brain, lung kidney, marrow…	Cell	Comes with UMAP coordinates; the brain reference does not apply to GBM malignant cells
scANVI / scArches	Semi-supervised deep model	Any atlas you train	Cell	Cross-technology, large scale; needs GPU

Naming: the label goes exactly as far as the evidence

```
new.ids <- c("0" = "Glial (undetermined)",  "1" = "Oligodendrocyte",
             "2" = "Glial (undetermined)",  "3" = "Glial (undetermined)",
             "4" = "Macrophage", "5" = "Macrophage", "6" = "Macrophage",
             "7" = "Cycling (undetermined)", "8" = "Microglia",
             "9" = "Pericyte / fibroblast", "10" = "T cell",
             "11" = "Glial (undetermined)", "12" = "Glial (undetermined)")
# 要刪哪一群由 02 的 QC 表決定，不要寫死編號
qc <- read.csv("output/tables/02_per_cluster_qc.csv")
new.ids[as.character(qc$cluster[qc$dbl > 0.5])] <- "DOUBLET"
gbm <- RenameIdents(gbm, new.ids); gbm$celltype <- Idents(gbm)
gbm <- subset(gbm, subset = celltype != "DOUBLET")
```

Number-to-name maps differ for every dataset — check your own DotPlot first; which cluster to drop comes from 02’s QC table, never a hard-coded number | Script 03 §4

Hierarchical annotation: major groups settled, now the subsets



Why recompute

1. Global HVGs are dominated by malignant-vs-immune; within-immune differences never make the top 2,000
2. Same for PCs — subset signal sits beyond PC 20
3. Rerun on immune cells alone and CD4/CD8, microglia vs blood-derived TAM separate

Take the immune major group out and rerun HVG, PCA and clustering — only then do CD4 / CD8 and microglia / blood-derived macrophages separate. Malignant 'subsets' use state scores, not clusters

Subsets: subset, then rerun the whole pipeline

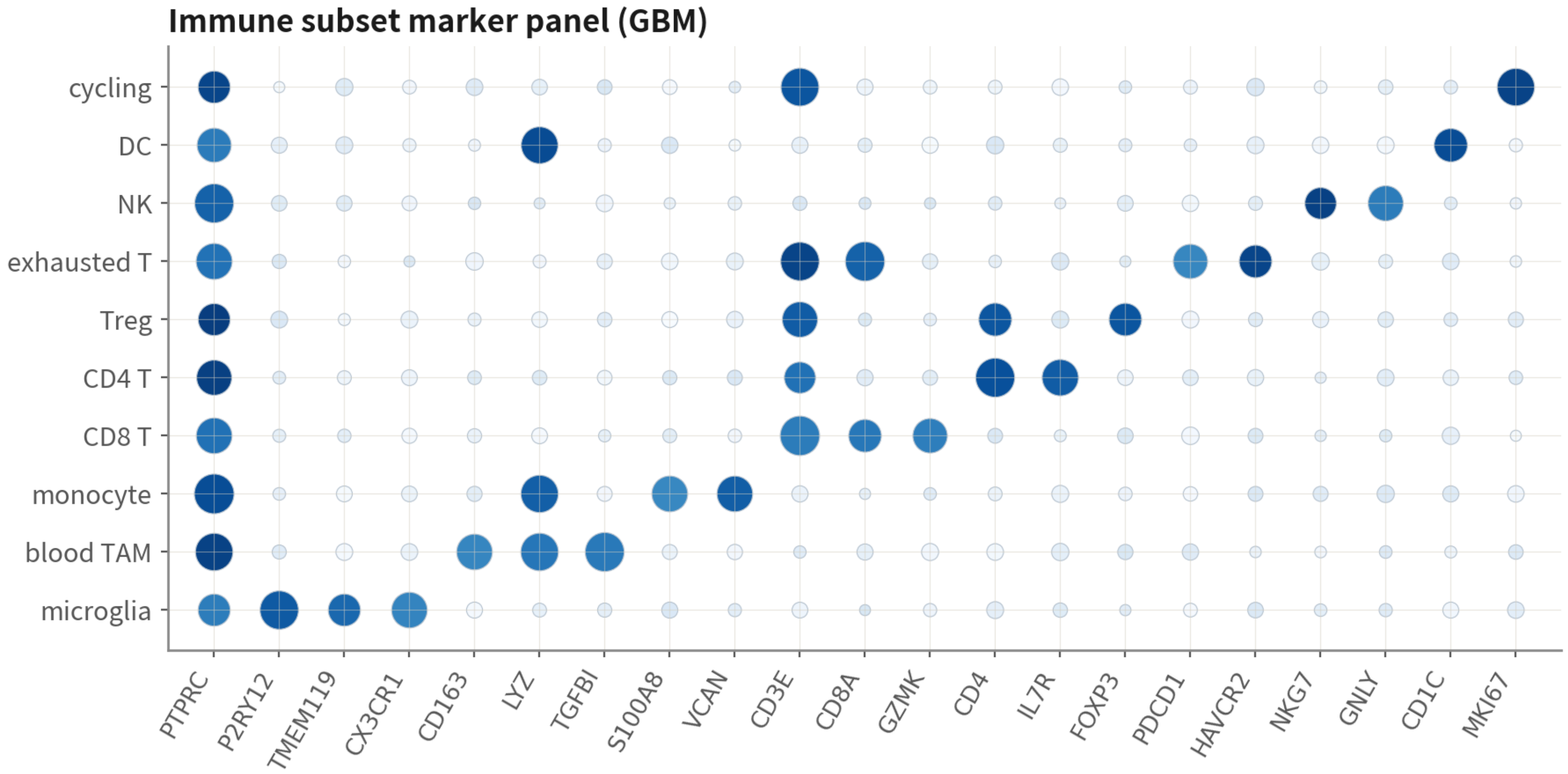
```r
imm <- subset(gbm, celltype %in% c("Macrophage", "Microglia", "T cell"))
imm <- NormalizeData(imm) |> FindVariableFeatures(nfeatures = 2000) |>
       ScaleData() |> RunPCA(npcs = 30)
ElbowPlot(imm)                                        # 亞群通常 15–20 個 PC 就夠
imm <- FindNeighbors(imm, dims = 1:20) |> FindClusters(resolution = 0.6) |>
       RunUMAP(dims = 1:20)
imm.panel <- c("P2RY12", "TMEM119", "CX3CR1",         # 小膠質
               "CD163", "LYZ", "TGFBI", "S100A8", "VCAN",   # 血液來源 TAM / 單核球
               "CD3E", "CD8A", "GZMK", "CD4", "IL7R", "FOXP3",
               "PDCD1", "HAVCR2", "NKG7", "GNLY", "CD1C", "MKI67")
DotPlot(imm, features = imm.panel) + RotatedAxis()
# 只想切某一群、不重跑：FindSubCluster(gbm, cluster = "2", graph.name = "RNA_snn")
```

Always recompute HVGs and PCA after subsetting; circling on the old UMAP is a common error. Do not split subsets under ~200 cells | Script 03 §4b

The immune subset panel: the diagonal is the evidence

Subset annotation: the dotplot diagonal is the evidence

schematic / simulated



Microglia vs blood-derived TAM, CD8 vs exhausted CD8, Treg — every cut needs its own markers; a group lit only by MKI67 is a state, merge it back

How to read

PTPRC lights the whole column: a gate, not a subset marker

Microglia vs blood TAM: P2RY12/TMEM119 vs CD163/TGFBI

Exhausted T=CD8 +PDCD1/HAVCR2, not a new type

Cycling immune lights only MKI67: a state—merge back

State scores for the glial clusters (Neftel 2019)

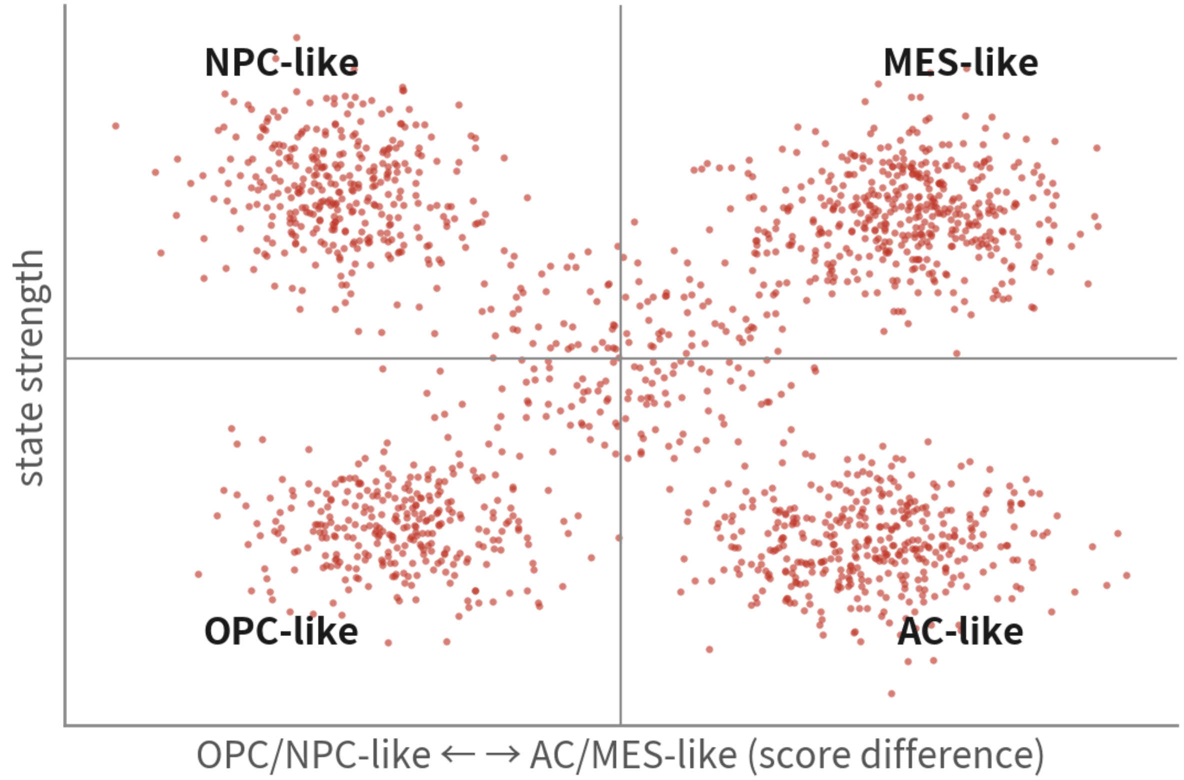
```
neftel <- list(                                    # 每組取前 8 個示範
  MES = c("CHI3L1", "ANXA2", "ANXA1", "CD44", "VIM", "MT2A"),
  AC  = c("CST3", "S100B", "SLC1A3", "HOPX", "MT3", "SPARCL1"),
  OPC = c("BCAN", "PLP1", "GPR17", "FIBIN", "LHFPL3", "OLIG1"),
  NPC = c("DLL3", "DLL1", "SOX4", "TUBB3", "HES6", "TAGLN3"))
glial <- subset(gbm, idents = "Glial (undetermined)")
glial <- AddModuleScore(glial, features = neftel, name = names(neftel))
FeaturePlot(glial, features = paste0(names(neftel), 1:4), ncol = 4)
```

Eight per set here; use the paper's supplementary table for the full sets (50 each). The states were defined inside confirmed-malignant cells — a score is never evidence of malignancy | Script 03 §5 | Deeper: B13

Why score states rather than cluster again

schematic / simulated

Four malignant states (Neftel 2019 gene sets, AddModuleScore)



Why score states instead of clustering again

- Malignant states are continuous; hard clusters invent boundaries
- Scoring with published gene sets makes results comparable across studies
- One patient's tumour holds all four states at once—one source of therapy resistance
- Score cell cycle (G2M/S) too, or cycling cells form their own cluster

Malignant states are continuous; scores keep that continuity and are comparable across studies.
Read the cycle score alongside

Three ways to score: AddModuleScore, UCell, AUCell

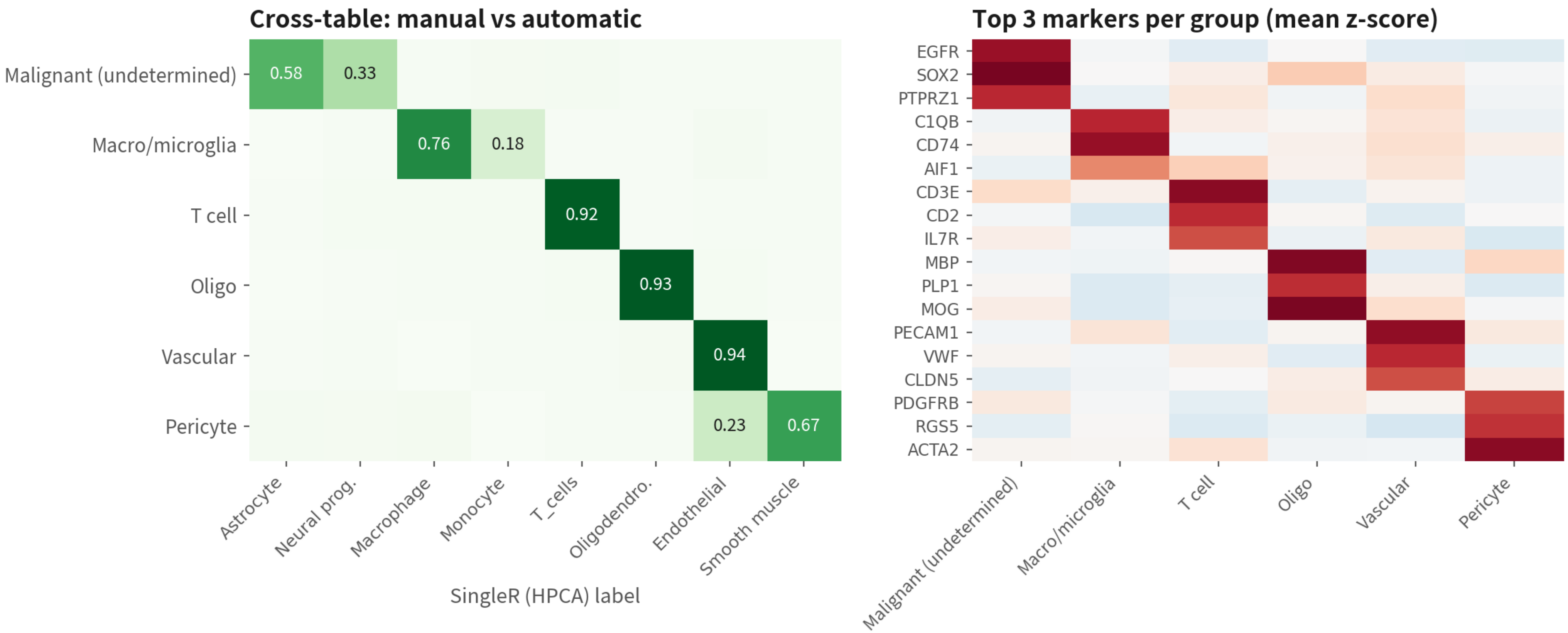
```
# 1. AddModuleScore（Seurat）：基因集平均 − 表現量相近的對照基因平均。快，但受深度影響
glial <- AddModuleScore(glial, features = neftel, name = names(neftel))

# 2. UCell：以基因在每顆細胞內的「排名」算 Mann–Whitney U 分數，對深度與稀疏穩健
library(UCell)
glial <- AddModuleScore_UCell(glial, features = neftel, name = "_UCell")

# 3. AUCell：每顆細胞的基因排名前 5% 裡有多少基因集成員（AUC）；SCENIC 也用它
library(AUCell)
rank <- AUCell_buildRankings(LayerData(glial, layer = "counts"))
auc  <- AUCell_calcAUC(neftel, rank)

FeaturePlot(glial, features = paste0(names(neftel), "_UCell"), ncol = 4)
cor(glial$MES1, glial$MES1_UCell)        # 三種算法的排序通常高度一致
```

Report both the algorithm and the source of the gene set; when the three disagree, trust UCell, which is least sensitive to depth | Script 03 §5

Annotation quality control: the cross-table and the marker heatmap

Two pieces of evidence after annotation: the cross-table and the marker heatmap

schematic / simulated



Left: a clean diagonal; malignant splitting into astrocyte/neural progenitor is expected (no tumour in the reference). Right: blocks along the diagonal, no group lighting everything

Left: a clean manual-vs-automatic diagonal with every deviation explained; right: top markers light in blocks, no group lights everything. Both go into the supplement

QC code: cross-table, heatmap, cluster tree, silhouette

```
# 交叉表：手動標籤 vs SingleR（每列加總為 1）
round(prop.table(table(gbm$celltype, gbm$singler), 1), 2)

# 每群 top 5 marker 熱圖（每群最多抽 100 顆，否則圖太重）
top5 <- markers |> group_by(cluster) |> slice_max(avg_log2FC, n = 5)
DoHeatmap(subset(gbm, downsample = 100), features = top5$gene, group.by = "celltype")

# 分群樹：型別之間的相似結構是否合理（免疫一支、膠質一支）
gbm <- BuildClusterTree(gbm, dims = 1:20); PlotClusterTree(gbm)

# 輪廓係數：每群在 PC 空間裡有多「自成一格」（< 0.1 的群要存疑）
library(cluster)
sil <- silhouette(as.integer(Idents(gbm)), dist(Embeddings(gbm, "pca")[, 1:20]))
tapply(sil[, "sil_width"], Idents(gbm), mean)
```

Four checks, four questions: does the automatic label agree, are markers clean, is the structure sensible, are clusters separable | Script 03 §5b

Naming convention: keep three label levels in metadata

Papers use level 2, supplementary tables level 3, statistical tests often only level 1

Column	Content	Example	Rule
celltype_l1	Major group	Immune / Malignant / Oligo / Vascular / Stromal	Five to seven; composition analysis uses this
celltype_l2	Type	Microglia, blood-derived TAM CD8 T, glial (undetermined)	Each backed by ≥ 2 markers; names aligned to Cell Ontology
celltype_l3	State / subset	Exhausted CD8 T, cycling TAM, MES-like	States as 'adjective + type'; do not coin new nouns
celltype_conf	Confidence	high / medium / unresolved	Write unresolved when unsure; never force

Common mis-annotations: five ‘novel types' and what they really are

Looks like	Actually	How to catch it	What to do
Tumour cells expressing immune genes	Malignant + macrophage doublets	High per-cluster doublet rate, high nCount, sits between two big groups	Remove the cluster and record it
A stem-like subset	A cycling existing type	Markers are only MKI67 / TOP2A; Phase nearly all S/G2M	Merge back, label 'cycling'
A stress-response subset	Dissociation artefact	FOS / JUN / HSPA1A high; a slice inside every type	Regress or annotate; not biology
Undifferentiated / unknown type	Low-quality cells	Lowest nFeature, highest mt, no marker lights	Back to QC
Immune cells Expressing oligodendrocyte genes	Ambient RNA	MBP / PLP1 a little in every group high pct.2	Rerun after SoupX

Deliverables: the three plots and one table of a Figure 1

```r
p1 <- DimPlot(gbm, group.by = "celltype", label = TRUE,
              repel = TRUE) + NoLegend()
p2 <- DotPlot(gbm, features = panel, group.by = "celltype") + RotatedAxis()
p3 <- VlnPlot(gbm, features = c("nFeature_RNA", "percent.mt"),
              group.by = "celltype", pt.size = 0, ncol = 2)
(p1 | p2) / p3                                 # patchwork 排版
ggsave("output/figs/03_fig1_annotation.pdf", width = 14, height = 9, bg = "white")

comp <- gbm@meta.data |> dplyr::count(celltype) |>
        mutate(pct = round(100 * n / sum(n), 1))
write.csv(comp, "output/tables/03_composition.csv", row.names = FALSE)
```

celltype	n	pct
Glial (undetermined)	2379	46.4
Macrophage	1178	23.0
Oligodendrocyte	850	16.6
Cycling (undetermined)	276	5.4
Microglia	171	3.3
Pericyte / fibroblast	140	2.7
T cell	136	2.7

UMAP for position, dotplot for evidence, violin for QC consistency, the table for numbers. Hand in all four and half the first-round questions disappear | Script 03 §6

Wrap-up: save the object, record the versions

```
saveRDS(gbm, "output/rds/03_gbm_annotated.rds")
write.csv(markers, "output/tables/03_markers.csv", row.names = FALSE)
sessionInfo()                          # 收進附錄；審稿人的好朋友
```

Every file in output/ can be regenerated from the scripts in R/ — that is what reproducible means | Script 03 §6

06

COMMON PITFALLS

The three most common hands-on traps in tumour data

Pitfall 1: copying the 5% mito cut

What it is Applying the PBMC tutorial‘s 5% to GBM.

Why it happens Every tutorial uses PBMC; 5% looks like common sense.

What it costs you Half the live cells gone, mostly active malignant ones. Immune proportions get inflated and every downstream composition analysis is skewed.

Pitfall 2: a doublet cluster reported as 'immune-like tumour cells'

What it is

A cluster expressing both SOX2 and CD74 written up as an ‘antigen-presenting malignant subset'.

Why it happens

The markers look exciting and there is literature to cite.

What it costs you

Sixty percent of the cluster is flagged by the doublet tool and it sits between malignant and myeloid. Sanity check 3 exists for exactly this.

Pitfall 3: cycling cells reported as a new type

What it is

An MKI67/TOP2A-high island named a 'stem-like subpopulation'.

Why it happens

Cell-cycle signal is strong and easily dominates clustering.

What it costs you

It is a state, not a type: dividing malignant cells and macrophages mixed together. Check the Phase column first, then decide whether to regress.

Q2 checklist

Q2 checklist: all nine boxes before the object goes to Q3

- ✓ raw object saved before filtering
- ✓ each QC cutoff has a number and a reason (MAD)
- ✓ doublet scores stored in metadata
- ✓ |cor(PC1, nCount)| < 0.3
- ✓ dims justified (elbow +quantified cutoff)
- ✓ resolution swept; stable band backed by markers
- ✓ four major groups settled with gate genes
- ✓ every name has two independent lines of evidence (markers+automatic)
- ✓ malignant label provisional: 'glial (undetermined)'

All nine boxes before the object goes to Q3

ONE-LINE SUMMARY

Every parameter is a hypothesis: a number, a reason, a record.
Annotation settles lineage first; malignancy awaits multiple lines of evidence.

Do this and your pipeline is reproducible and defensible. Calling malignancy needs evidence one sample cannot give; Q3 switches datasets to show it.

Check yourself 1

GBM: median percent.mt 7%, median + 3×MAD = 13%. Where does the cutoff go?

A The conventional 5%

B 13%, with the number and reason in the methods

C Skip the percent.mt filter

Answer B. A threshold is a property of the data. 5% removes most live cells; no filter keeps the dying-cell mode.

Check yourself 2

A cluster expresses both SOX2 and C1QB; the doublet tool flags 65% of it. What now?

A An exciting new subset — report it

B Remove the whole cluster and record why

C Remove only the flagged 65%

Answer B. Mostly flagged and sitting between two big groups — the whole cluster is a doublet product; the remaining 35% are the ones the tool missed, not real cells.

Check yourself 3

SingleR labels cluster 0 as Astrocyte; markers are SOX2 / PTPRZ1 / EGFR. The honest label?

A Astrocyte

B Malignant

C Glial (malignancy undetermined)

Answer C. No GBM malignant cells exist in the reference, so SingleR picks the nearest; markers only give lineage. Malignancy needs genome-level evidence plus patient-specificity — one patient cannot supply both, so undetermined is the honest label.

NEXT

Q3: multi-sample integration and downstream analysis

Four patients, core vs periphery (GSE84465) — the dataset changes precisely to supply what one sample cannot. Malignancy is called from multiple lines of evidence led by inferCNV, pseudobulk puts DE at the patient level, and the episode closes with a downstream routing guide.

https://github.com/Charlene717/scRNA-seq-tutorial-Qseries